CURRENT STATE STUDY GUIDE

state of sports analysis platform as of 2026-04-21

A founder and principal engineer reading document built from the vault and the live code path. This captures the implemented state at the end of 2026-04-21.

STATE BOUNDARY

2026-04-21

PRIMARY AUDIENCE

Founder / Principal Engineer

SOURCE TYPES

Vault docs plus live Angular, .NET, and Python code

CONTENTS

1. executive summary
2. current product stance after analyze
3. supported competitions and grounded evidence
4. the simplest mental model
5. what moves where
6. major moving parts and ownership
7. major architectural decisions made so far
8. what the system stores and returns after analyze
9. important caveats and risks
10. recommended study order

11. the current-state story in one paragraph
12. appendix: key questions this document should help you answer

---

derived from:

- vault docs in `dai-vault/02 Platform/architecture/`, `dai-vault/04 Products/sports-v1/`, and `dai-vault/06 Execution/`
- live code in `dai/apps/sports-app/`, `dai/platform/dotnet/`, and `dai/services/agent-service/`

purpose:

- give a founder / principal engineer study guide for the sports analysis stack
- explain the current state at the end of 2026-04-21, not the future target state
- show both the high-level flow and the specific architectural decisions that matter

boundary note:

- this document intentionally describes the implemented state as of 2026-04-21
- later work should be compared against this baseline, not mixed into it

---

---

executive summary

as of 2026-04-21, the sports product is a real on-demand, competition-aware pregame analysis engine with a thin angular shell around it.

the system is no longer "frontend calls model and gets a summary." it now has a clearer platform shape:

1. angular collects the matchup selection and sends a typed analysis request
2. .net stores an `AgentRun`, collects grounded evidence, calls python, then calibrates the final confidence
3. python receives only the data the platform actually retrieved, writes the `lean`, `summary`, and `factors`, and returns a provisional confidence

4. .net persists the output and returns a trimmed user-facing response

the most important architectural change so far is that the model is no longer treated as the source of truth for everything. the platform is starting to own:

- competition routing
- grounded signal provenance
- confidence calibration
- cross-layer traceability
- durable run records

that is the right direction if the long-term product is decision intelligence rather than report generation.

---

current product stance after analyze

today's product stance is narrower than the long-term vault vision.

what is real now:

- the analyzer page is real and wired end to end
- the app is competition-aware, not just sport-aware
- analysis requests travel through angular -> .net -> python -> .net -> angular
- collector evidence is attached before the model call
- final confidence is calibrated in .net after the model call
- every run is stored in SQL as an `AgentRun`

what the user gets after clicking `Analyze`:

- a `lean`
- a `summary`
- a calibrated `confidence`
- a list of `factors`

- an `agentRunId`
- server-side runtime metadata

what it is not yet:

- not yet a scheduled delivery product
- not yet a billing-gated product
- not yet a full history product
- not yet a learning-loop product
- not yet a structured signal-table product with source provenance shown in the UI

practical reading:

- this is a working analysis engine and dev surface
- it is not yet the full commercial sports business system

---

supported competitions and grounded evidence

the internal routing key is now an explicit competition code.

current supported competitions:

- `nfl`

  - family: football - level: pro - grounded evidence today: current market spread

- `ncaaf`

  - family: football - level: college - grounded evidence today: current market spread

- `nba`

  - family: basketball - level: pro - grounded evidence today: rest / schedule context plus current market spread

- `ncaamb`

- family: basketball - level: college - grounded evidence today: rest / schedule context plus current market spread

- `mlb`

  - family: baseball - level: pro - grounded evidence today: probable starters

- college baseball

  - visible in the UI as unavailable - intentionally not implemented

live code excerpt from `CompetitionCatalog.cs`:

```
public static class CompetitionCatalog
{
    private static readonly CompetitionDefinition[] _all =
    [
        new(
            Code: CompetitionCodes.Nfl,
            DisplayName: "NFL",
            SportFamily: "football",
            Level: "pro",
            OddsApiKey: "americanfootball_nfl",
            MaxGroundedSignals: 1),
        new(
            Code: CompetitionCodes.Ncaaf,
            DisplayName: "NCAAF",
            SportFamily: "football",
            Level: "college",
            OddsApiKey: "americanfootball_ncaaf",
            MaxGroundedSignals: 1),
        new(
            Code: CompetitionCodes.Nba,
            DisplayName: "NBA",
            SportFamily: "basketball",
            Level: "pro",
            OddsApiKey: "basketball_nba",
            MaxGroundedSignals: 2),
        new(
            Code: CompetitionCodes.Ncaamb,
            DisplayName: "NCAAMB",
            SportFamily: "basketball",
            Level: "college",
            OddsApiKey: "basketball_ncaab",
            MaxGroundedSignals: 2),
        new(
            Code: CompetitionCodes.Mlb,
            DisplayName: "MLB",
            SportFamily: "baseball",
            Level: "pro",
            OddsApiKey: "baseball_mlb",
            MaxGroundedSignals: 1),
        new(
            Code: null,
```

```
            DisplayName: "College Baseball",
            SportFamily: "baseball",
            Level: "college",
            AvailabilityNote: "College baseball is not available yet.",
            OddsApiKey: null,
            MaxGroundedSignals: 0)
    ];
```

why this matters:

- the UI stays friendly with sport + level selection
- the platform gets explicit routing identity
- collector logic, evaluator logic, and seed data can now stay honest

---

---

the simplest mental model

if you want a founder-level shorthand, use this:

- angular is the packaging layer
- .net is the factory floor and traffic cop
- python is the writer / synthesizer
- SQL is the memory
- external sports APIs are the raw materials

another useful framing:

- platform = factory
- agents = workers
- frontends = packaging
- stripe = truth

in the sports slice, .net is currently acting as the factory coordinator. python is one worker in that factory, not the whole factory.

---

what moves where

before analyze

before the user clicks `Analyze`, the UI is still building context.

the sequence is:

1. angular loads visible competitions from `GET /api/competitions`
2. angular loads teams from `GET /api/competitions/{code}/teams`
3. angular loads real matchup dates from `GET /api/competitions/{code}/matchup-dates?teamA=...&teamB=...`
4. .net resolves the real event orientation and returns `homeTeam` / `awayTeam` for the selected date

important boundary:

- this part does **not** go through python
- python is only used for the actual analysis call

the reference endpoint is validating competition and team selection inside `.NET`, not inside the model path.

live code excerpt from `SportsReferenceController.cs`:

```
public async Task<ActionResult<IReadOnlyList<CompetitionReferenceDto>>>
GetCompetitions(CancellationToken ct)
{
    var activeCodes = await db.Sports
        .Where(s => s.IsActive)
        .Select(s => s.Slug)
        .ToListAsync(ct);

    var activeSet = activeCodes.ToHashSet(StringComparer.OrdinalIgnoreCase);

    var competitions = CompetitionCatalog.All
        .Where(c => c.Code is null || activeSet.Contains(c.Code))
        .Select(c => new CompetitionReferenceDto(
```

and the matchup-date path is explicitly validated:

```
public async Task<ActionResult<IReadOnlyList<MatchupEventDto>>> GetMatchupDates(
    string code,
    [FromQuery] string? teamA,
    [FromQuery] string? teamB,
    CancellationToken ct)
{
    var normalized = code.ToLowerInvariant();

    if (!CompetitionCatalog.TryGetSupported(normalized, out _))
    {
        return NotFound();
    }

    if (string.IsNullOrWhiteSpace(teamA) || string.IsNullOrWhiteSpace(teamB))
    {
        return BadRequest("teamA and teamB are required");
    }
```

when analyze is clicked

once a user has selected a real event, the actual analysis pipeline starts.

high-level flow:

```
Angular
  -> POST /api/agent-runs
.NET AgentRunsController
  -> create pending AgentRun row
  -> AgentRunService.ExecuteSportsMatchupAsync()
     -> CollectAsync()
     -> FastApiClient.AnalyzeSportsMatchupAsync()
     -> Evaluate()
     -> ComposeDecisionArtifact()
  -> update AgentRun row to completed / failed
Angular
  <- lean, summary, confidence, factors, runtime metadata
```

the angular payload boundary

angular is responsible for packaging the request, but it does not decide final home / away orientation from the user's team order. it uses the selected event returned by the schedule lookup.

live code excerpt from `analyzer.component.ts`:

```
analyze(): void {
  this.error.set(null)
  this.result.set(null)
  this.analyzedMatchup.set(null)

  if (this.form.invalid) {
    this.form.markAllAsTouched()
    return
  }

  const ev = this.selectedEvent()
  if (!ev) return

  const competition = this.form.getRawValue().competition

  this.loading.set(true)
  this.form.disable({ emitEvent: false })

  this.api.createMatchupAnalysis({
    runType: 'sports.matchup.analysis',
    agentProfileId: null,
    input: {
      competition,
      // use resolved orientation from the selected event, not the user's Team
A/B selection order
      homeTeam: ev.homeTeam,
      awayTeam: ev.awayTeam,
      gameDate: ev.date
    }
  }).subscribe({
```

why this matters:

- the user chooses a neutral team pair
- the platform chooses the actual provider-resolved home / away matchup
- spread matching and collector lookups become deterministic

the .net execution pipeline

the current orchestration shape is a three-step pipeline:

1. collect grounded evidence
2. analyze with python
3. evaluate and calibrate confidence

live code excerpt from `AgentRunService.cs`:

```
private async Task<AgentRunExecutionResult> ExecuteSportsMatchupAsync(
    CreateAgentRunRequest req, Guid agentRunId, CancellationToken ct)
{
    var gameDate = req.Input.GameDate.ToString("yyyy-MM-dd");

    // -- step 1: collect --
    var collector = await CollectAsync(req.Input, gameDate, ct);

    // -- step 2: analyze --
    var aiReq = new SportsAnalysisRequest(
        Competition:               req.Input.Competition,
        HomeTeam:                  req.Input.HomeTeam,
        AwayTeam:                  req.Input.AwayTeam,
        GameDate:                  gameDate,
        FootballMarketContext:     collector.FootballMarketContext,
        MlbStarterContext:         collector.MlbStarterContext,
        BasketballScheduleContext: collector.BasketballScheduleContext,
        BasketballMarketContext:   collector.BasketballMarketContext
    );

    var aiRes = await ai.AnalyzeSportsMatchupAsync(aiReq, agentRunId, ct);

    // -- step 3: evaluate --
    var evaluator = Evaluate(aiRes, collector, req.Input.Competition);

    return ComposeDecisionArtifact(aiRes, collector, evaluator);
}
```

why this matters:

- python is no longer the only step

- evidence retrieval happens before the model call
- final confidence is a platform concern, not only a model concern

the python routing boundary

python validates supported competitions and dispatches to the right analyzer family.

live code excerpt from `services/agent-service/app/routes/sports.py`:

```
async def analyze_sports(req: SportsAnalysisRequest, request: Request) ->
SportsAnalysisResponse:
    competition = req.competition.lower()
    correlation_id = request.headers.get("x-correlation-id", "none")
    agent_run_id = request.headers.get("x-agent-run-id", "none")

    if competition not in _SUPPORTED_COMPETITIONS:
        raise HTTPException(
            status_code=400,
            detail=f"competition '{req.competition}' is not supported --
supported competitions: {', '.join(sorted(_SUPPORTED_COMPETITIONS))}",
        )

    try:
        if competition in {"nfl", "ncaaf"}:
            return await analyze_football(req.homeTeam, req.awayTeam,
req.gameDate, competition, req.footballMarketContext)
        if competition in {"nba", "ncaamb"}:
            return await analyze_basketball(
                req.homeTeam,
                req.awayTeam,
                req.gameDate,
                competition,
                req.basketballScheduleContext,
                req.basketballMarketContext,
            )
        return await analyze_mlb(req.homeTeam, req.awayTeam, req.gameDate,
req.mlbStarterContext)
```

why this matters:

- python receives an already-resolved competition
- family-level prompt behavior stays clean
- unsupported competition rejection is explicit, not prompt-driven

grounded signal ownership

the collector layer, not python, owns the statement "what real signals existed for this run."

live code excerpt from `SportsCollectorOutput.cs`:

```
public SportsCollectorOutput(
    FootballMarketContext? footballMarketContext,
    MlbStarterContext? mlbStarterContext,
    BasketballScheduleContext? basketballScheduleContext,
    BasketballMarketContext? basketballMarketContext)
{
    FootballMarketContext     = footballMarketContext;
    MlbStarterContext         = mlbStarterContext;
    BasketballScheduleContext = basketballScheduleContext;
    BasketballMarketContext   = basketballMarketContext;

    var signals = new List<string>(4);
    if (footballMarketContext is not null) signals.Add("market");
    if (mlbStarterContext is not null) signals.Add("starting_pitching");
    if (basketballScheduleContext is not null) signals.Add("rest_schedule");
    if (basketballMarketContext is not null) signals.Add("market");
    GroundedSignals = [.. signals];
}
```

why this matters:

- grounded provenance stays platform-owned
- the model cannot silently claim evidence richness it did not have
- future learning loops can compare outcome accuracy against actual grounded categories

confidence calibration

the analyzer's confidence is treated as provisional. the final displayed confidence is calibrated in `.NET` based on grounded evidence richness.

live code excerpt from `AgentRunService.cs`:

```
private static EvaluatorOutput Evaluate(
    SportsAnalysisResponse analyzerOutput,
    SportsCollectorOutput  collector,
    string                 competition)
{
    var maxGrounded   = CompetitionMaxGroundedSignals(competition);
    var groundedCount = Math.Min(collector.GroundedSignals.Length, maxGrounded);

    var calibration = ResolveConfidenceCalibration(groundedCount, maxGrounded);
    var calibrated = Math.Clamp(
        analyzerOutput.Confidence * calibration.Dampening,
        calibration.MinConfidence,
        calibration.MaxConfidence);

    var band = calibrated >= 0.70 ? "high"
             : calibrated >= 0.50 ? "medium"
             : "low";

    return new EvaluatorOutput(
        AggregateConfidence: calibrated,
        AnalyzerConfidence:  analyzerOutput.Confidence,
        ConfidenceBand:      band
    );
}
```

what this means in practice:

- `0 grounded` is treated as priors-only reasoning
- basketball can now distinguish `0 of 2`, `1 of 2`, and `2 of 2`
- the platform is beginning to separate "the model sounded confident" from "the system had real evidence"

cross-layer tracing

the stable cross-layer trace anchor is `X-Agent-Run-Id`.

live code excerpt from `FastApiClient.cs`:

```
public async Task<SportsAnalysisResponse>
AnalyzeSportsMatchupAsync(SportsAnalysisRequest req, Guid agentRunId,
CancellationToken ct)
{
    var request = new HttpRequestMessage(HttpMethod.Post, "/api/sports/analyze");
    request.Headers.Add("X-Correlation-Id", GetCorrelationId());
    // forward the stable agentRunId so fastapi logs can be tied back to the
agentrun db row
    request.Headers.Add("X-Agent-Run-Id", agentRunId.ToString());
    request.Content = JsonContent.Create(req);
```

why this matters:

- angular shows the run id after completion
- SQL stores the run
- python logs can be tied back to the same run
- debugging becomes much easier than relying on separate trace formats

---

---

major moving parts and ownership

angular sports app

owns:

- user interaction
- sport / level / team / date selection
- neutral team pair selection
- displaying the final result
- showing `agentRunId` and runtime metadata in the UI

does not own:

- evidence retrieval
- final confidence calibration

- sport-family business logic beyond form flow

key files:

- `dai/apps/sports-app/src/app/analyzer/analyzer.component.ts`
- `dai/apps/sports-app/src/app/sports-api.service.ts`
- `dai/apps/sports-app/src/app/core/models/agent-run.model.ts`
- `dai/apps/sports-app/src/app/app.routes.ts`

.net platform api

owns:

- competition-aware routing
- SQL-backed reference data
- creating and updating `AgentRun`
- collector orchestration
- transport to python
- evaluator logic
- final decision artifact assembly

does not own:

- natural-language synthesis
- long narrative phrasing

key files:

- `dai/platform/dotnet/DevCore.Api/Controllers/SportsReferenceController.cs`
- `dai/platform/dotnet/DevCore.Api/Controllers/AgentRunsController.cs`
- `dai/platform/dotnet/DevCore.Api/AgentRuns/AgentRunService.cs`
- `dai/platform/dotnet/DevCore.Api/AgentRuns/SportsCollectorOutput.cs`
- `dai/platform/dotnet/DevCore.Api/Sports/CompetitionCatalog.cs`

collector clients in .net

owns:

- fetching raw grounded evidence before the model call

current clients:

- `OddsScheduleClient`

  - matchup dates and resolved home / away orientation

- `OddsMarketClient`

  - football and basketball current spread

- `EspnBasketballScheduleClient`

  - basketball rest / schedule grounding

- `MlbStarterClient`

  - probable starters

important principle:

- collectors are the raw-material side of the factory
- they should stay deterministic and thin

python agent service

owns:

- family-based prompt selection
- model call to OpenAI
- writing the `lean`, `summary`, and `factors`
- returning provisional analyzer confidence

does not own:

- grounded signal provenance
- final displayed confidence
- SQL persistence

key files:

- `dai/services/agent-service/app/routes/sports.py`
- `dai/services/agent-service/app/models/sports.py`
- `dai/services/agent-service/app/services/sports_analyzer.py`

SQL persistence

owns:

- reference data for sports / competitions and teams
- durable `AgentRun` records
- stored input and output payloads

important business role:

- this is the memory layer for debugging, future history views, and eventual learning loops

external providers

current external dependencies:

- The Odds API

  - schedule lookup - current spread lookup

- ESPN public scoreboard

  - basketball rest / schedule context

- MLB Stats API

  - probable starters

- OpenAI via python

  - language synthesis

important product truth:

- the current architecture is ahead of the current evidence depth
- the evidence set is still intentionally thin

---

major architectural decisions made so far

1. competition-first routing instead of sport-first routing

decision:

- move from vague sport routing to explicit competition codes like `nfl`, `ncaaf`, `nba`, `ncaamb`, `mlb`

why it was made:

- pro and college variants need different seed data, schedule keys, collector behavior, and confidence richness

founder implication:

- the product can stay user-friendly while the platform stays explicit and scalable

2. resolve home / away from a real scheduled event, not from user order

decision:

- the selected event from schedule lookup determines `homeTeam` and `awayTeam`

why it was made:

- user selection order is not reliable enough for spread matching or provider consistency

founder implication:

- correctness was chosen over a superficially simpler input model

3. introduce a real collect -> analyze -> evaluate pipeline

decision:

- stop treating the model call as the whole pipeline

why it was made:

- evidence retrieval, reasoning, and confidence calibration are different responsibilities

founder implication:

- the system is moving toward reusable platform roles rather than prompt-only product logic

4. keep grounded signal ownership in the collector layer

decision:

- `GroundedSignals` is computed in `.NET`, not in python

why it was made:

- the platform can know what it retrieved
- the model cannot reliably self-report that

founder implication:

- this is foundational for honest provenance and future outcome tracking

5. make evaluator confidence the displayed confidence

decision:

- the model's confidence is stored, but the evaluator's calibrated confidence becomes the user-facing `confidence`

why it was made:

- narrative confidence is not the same as evidence-backed confidence

founder implication:

- this is the correct design move if you eventually want trust, calibration, and measurable signal quality

6. make python fail closed when collector data is missing

decision:

- if schedule, market, or starter data is missing, python gets an explicit no-data instruction and omits that factor

why it was made:

- reduce hallucinated claims

founder implication:

- the system is choosing honesty over prettier but less reliable output

7. treat `AgentRun` as the audit spine

decision:

- create the run row before execution, store input, persist output, and preserve failures

why it was made:

- failed and successful runs both need durable traceability

founder implication:

- this is the right substrate for support, debugging, history, and future analytics

8. stay thin and resist premature abstraction

decision:

- do not build the full orchestrator / profile / scoring framework before there are enough real pipeline differences to justify it

why it was made:

- premature platformization would outrun product proof

founder implication:

- current code intentionally contains some competition branches in `.NET`
- that is acceptable because the product is still proving the slice

---

---

what the system stores and returns after analyze

what angular receives

the main UI response is intentionally small:

```
{
  "agentRunId": "...",
  "status": "completed",
  "lean": "edge toward ...",
  "summary": "...",
  "confidence": 0.64,
  "factors": ["...", "...", "..."],
  "createdUtc": "...",
  "durationMs": 1842
}
```

meaning:

- the UI gets the main artifact it needs to render
- the UI does not get raw collector payloads
- the UI does not yet get grounded signal provenance directly

what SQL stores

the stored `AgentRun` row is richer than the UI response.

it includes:

- `RunType`
- `Status`
- `InputJson`
- `OutputJson`
- `ErrorMessage`
- `CorrelationId`
- `StartedUtc`
- `CompletedUtc`
- `DurationMs`

and `OutputJson` now includes more than the UI shows:

- `lean`
- `summary`
- calibrated `confidence`
- `factors`
- `groundedSignals`
- `analyzerConfidence`

important implication:

- the stored output is already closer to a real decision artifact than the UI surface is

current product reality after analyze

after `Analyze`, the user has a useful read, but not the full future product experience.

they do not yet have:

- persistent real history in the UI
- delivery status
- source citations in the UI
- structured signal tables
- outcome tracking

this is why the current result should be understood as a **proxy decision artifact**, not the finished artifact model described in the vault's longer-term architecture docs.

---

important caveats and risks

production build still points at stub api behavior

local dev serve uses the development environment, which is live:

```
export const environment = {
  production: false,
  useStubApi: false,
  useStubSchedule: false,
  apiBaseUrl: 'http://localhost:5007'
}
```

but the default `environment.ts` still enables stub api behavior:

```
export const environment = {
  production: true,
  useStubApi: true,
  useStubSchedule: false,
  apiBaseUrl: 'http://localhost:5007'
}
```

why this matters:

- `ng serve` should hit the live stack
- a production-style build can still short-circuit inside angular if this is not corrected

history and account are shell surfaces

current truth:

- analyzer is real
- history is mock data
- account is static placeholder data

why this matters:

- the app shell is ahead of the real product surface behind those routes

auth is not production-grade yet

current truth:

- identity resolution supports dev bypass

- this is acceptable for current development, not for real tenant-facing production

evidence depth is still thin

current grounded evidence:

- football: market only
- basketball: rest / schedule + market
- baseball: starters only

not yet grounded:

- injuries
- weather
- line movement history
- public / sharp money
- richer situational models

why this matters:

- the architecture is getting stronger faster than the signal depth

`OutputJson` is still blob storage

current truth:

- the platform stores structured meaning inside a raw JSON string column

why this matters:

- good enough for now
- weak for structured analytics later

some docs are newer than others

most trustworthy current-state docs:

- `current-sports-analysis-flow.md`
- `orchestration.md`

- `current-sports-matchup-analyzer.md`

older but still useful:

- `current-agent-run-contract.md`

why this matters:

- some contract docs lag the most recent basketball grounding changes

---

---

recommended study order

pass 1: founder / product understanding

read these first:

1. `dai-vault/02 Platform/architecture/current-sports-analysis-flow.md`
2. `dai-vault/02 Platform/architecture/orchestration.md`
3. `dai-vault/06 Execution/handoffs/current-sports-matchup-analyzer.md`
4. `dai-vault/04 Products/sports-v1/ui-concept.md`

goal:

- understand what the product is today
- understand why the pipeline shape exists
- understand what is real vs shell

pass 2: code path understanding

read these second:

1. `dai/apps/sports-app/src/app/analyzer/analyzer.component.ts`
2. `dai/apps/sports-app/src/app/sports-api.service.ts`
3. `dai/platform/dotnet/DevCore.Api/Controllers/SportsReferenceController.cs`
4. `dai/platform/dotnet/DevCore.Api/Controllers/AgentRunsController.cs`
5. `dai/platform/dotnet/DevCore.Api/AgentRuns/AgentRunService.cs`

6. `dai/platform/dotnet/DevCore.Api/AgentRuns/SportsCollectorOutput.cs`
7. `dai/platform/dotnet/DevCore.AiClient/FastApiClient.cs`
8. `dai/services/agent-service/app/routes/sports.py`
9. `dai/services/agent-service/app/services/sports_analyzer.py`

goal:

- trace the live request path end to end
- see where ownership changes across layers

pass 3: operational debugging understanding

focus on:

- `agentRunId`
- `InputJson`
- `OutputJson`
- collector branches by competition
- environment split between live dev and stub production config

goal:

- be able to answer "what happened in this run?" with confidence

---

---

the current-state story in one paragraph

the sports platform as of 2026-04-21 is a thin but credible decision-engine foundation. angular is a competition-aware shell that packages a real matchup request. .net is the authoritative platform layer that validates competition selection, retrieves grounded evidence, stores runs, and calibrates final confidence. python is a constrained synthesis worker that writes the natural-language read from the evidence it is given. SQL is the memory layer. the product is still early and the evidence set is still thin, but the major architectural decisions made so far are pointed in the right direction: competition-first routing, platform-owned provenance, evaluator-owned confidence, and durable run traceability.

---

appendix: key questions this document should help you answer

- what is real in the product today versus still mocked?
- where does the request leave angular?
- which parts go through python and which parts do not?
- who decides what evidence was actually grounded?
- who owns final confidence?
- what exactly gets stored after a run?
- what are the biggest current risks if this were pushed toward production too early?

Generated from the vault markdown source so the study doc stays maintainable. If implementation changes after 2026-04-21, update the markdown first and then regenerate the print companion.